

Cloud Native

1. Overview

Cloud Native is a way of designing, building, deploying, and operating applications to fully leverage cloud environments.

It focuses on achieving key non-functional requirements such as:

- Scalability
- Fault tolerance
- High availability
- Maintainability
- Fast deployments
- Observability
- Automation

Cloud native is **not just hosting on cloud**. It is an architecture and operating model.

2. Simple Definition

Traditional thinking:

None

Take old app

Move VM to cloud

Cloud Native thinking:

None

Design app specifically for elastic, automated, distributed cloud environments

3. Core Goals

Cloud native systems should be:

- Resilient
- Manageable
- Observable
- Loosely coupled
- Easy to scale
- Easy to change frequently

4. Key Building Blocks

Common cloud native technologies:

1. Containers
2. Microservices
3. Service Mesh
4. Immutable Infrastructure
5. Declarative APIs
6. Automation / CI-CD
7. Serverless
8. Observability

5. Containers

Package app + dependencies consistently.

Popular platform:

- Docker

Benefits:

- Portable

- Fast startup
- Consistent environments
- Efficient resource usage

6. Orchestration

Run many containers automatically.

Popular platform:

- Kubernetes

Handles:

- Scheduling
- Scaling
- Self-healing
- Rolling deploys
- Service discovery

7. Microservices

Split system into smaller independently deployable services.

Example:

None

User Service

Order Service

Payment Service

Notification Service

Benefits:

- Independent scaling
- Faster releases
- Team autonomy

Trade-off:

- More operational complexity

8. Service Mesh

Manages service-to-service communication.

Examples:

- Istio
- Linkerd

Provides:

- mTLS
- Retries
- Traffic routing
- Observability
- Policy enforcement

9. Immutable Infrastructure

Do not manually patch running servers.

Instead:

None

Build new image

Deploy new instances

Destroy old instances

Benefits:

- Predictable deployments
- No config drift
- Easier rollback

10. Infrastructure as Code (IaC)

Infrastructure managed as code.

Examples:

- Terraform
- AWS CloudFormation

Benefits:

- Version control
- Repeatability
- Auditability
- Faster environment setup

11. Declarative APIs

Tell system desired state, not manual steps.

Example:

None

```
replicas = 5
```

Platform decides how to achieve it.

Common in Kubernetes.

12. Automation

Cloud native strongly favors automation:

- CI/CD
- Auto scaling
- Auto healing
- Security scanning
- Backup jobs
- Policy checks

Reduces manual toil.

13. Serverless

Run functions without managing servers.

Examples:

- Amazon Web Services AWS Lambda
- Google Cloud Functions

Good for:

- Event-driven workloads
- Burst traffic
- Low ops overhead

14. Observability

Cloud native systems need deep visibility.

Three pillars:

- Logs
- Metrics
- Traces

Tools:

- Prometheus
- Grafana
- Jaeger

15. Why Cloud Native Helps NFRs

Requirement	Cloud Native Benefit
Scalability	Autoscaling containers/services
Fault tolerance	Self-healing + replicas
Maintainability	CI/CD + small services
Availability	Multi-zone deployments
Security	Policy automation + mTLS
Speed	Fast deployments

16. Example Architecture



Autoscaling + rolling deploy supported.

17. Traditional vs Cloud Native

Aspect	Traditional	Cloud Native
Deployments	Manual	Automated
Scaling	Vertical/manual	Horizontal/auto

Infra changes	Ticket/manual	Code driven
Releases	Slow	Frequent
Failure handling	Manual	Self-healing

18. Cloud Native Design Principles

- Stateless services when possible
- Horizontal scale first
- Failure expected, design resilience
- Automate repetitive work
- Use managed services wisely
- Monitor everything important

19. Trade-offs / Challenges

More moving parts

Learning curve

Kubernetes complexity

Distributed debugging harder

Cost sprawl if unmanaged

Cloud native is powerful, not automatically simple.

20. When NOT to Overdo It

Small internal app with 20 users may not need:

None

Kubernetes

Service mesh

20 microservices

Simple monolith on VM may be better.

21. Interview Answer Example

Q: How would you design scalable modern backend?

Answer:

I'd use a cloud-native approach: containerized stateless services, orchestration for autoscaling and self-healing, CI/CD pipelines, managed databases where suitable, observability tooling, and infrastructure as code for repeatable operations.

22. Best Practices

- Start simple, evolve gradually
- Use containers consistently
- Prefer stateless APIs
- Use IaC for all environments
- Automate deployments
- Implement health checks
- Use observability from day one
- Avoid unnecessary microservices early

23. Common Tools Ecosystem

- Docker
- Kubernetes
- Helm
- Terraform
- Argo CD
- Prometheus

24. Short Revision Notes

None

Cloud Native = Build apps for cloud environments

Uses:

- Containers
- Kubernetes
- Microservices
- Service mesh
- IaC
- CI/CD
- Observability

Benefits:

- Scalable
- Resilient
- Fast releases
- Maintainable

25. One-Line Summary

Cloud native is an architecture approach that uses containers, automation, scalable services, and declarative infrastructure to build resilient, maintainable, and rapidly evolving systems in cloud environments.